LAB MANUAL

Course Code: 24UADPC303
Course Title: Deep Learning with Keras and TensorFlow Laboratory

<table>
<tr><th>Exp. No.</th><th>Laboratory Experiment/Practical Titles</th><th>Relevant COs Number (s)</th></tr>
<tr><td>1</td><td>Build a deep neural network model start with linear regression using a single variable.

Aim

To implement linear regression using TensorFlow and Keras with a single independent variable.

Theory

Linear Regression is a supervised learning algorithm used to predict a continuous value. It establishes a relationship between an input variable (X) and an output variable (Y).

Program
<pre>import tensorflow as tf
import numpy as np

X = np.array([1,2,3,4,5], dtype=float)
Y = np.array([2,4,6,8,10], dtype=float)

model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=1, input_shape=[1])
])

model.compile(optimizer='sgd', loss='mean_squared_error')

model.fit(X, Y, epochs=500, verbose=0)

print(model.predict(np.array([6.0])))</pre>
Sample Output
<pre>[[11.98]]</pre></td><td>COPC 303.1</td></tr>
<tr><td>2</td><td>Build a deep neural network model start with linear regression using multiple variables.

Title

Build a Deep Neural Network Model Starting with Linear Regression Using Multiple Variables

CO Mapping: COPC303.1</td><td>COPC 303.1</td></tr>
</table>

Aim

To implement multiple linear regression using TensorFlow and Keras.

Theory

Multiple Linear Regression predicts a target value using multiple input variables.

Program

```
import tensorflow as tf
import numpy as np

X = np.array([
    [1,2],
    [2,3],
    [3,4],
    [4,5],
    [5,6]
], dtype=float)

Y = np.array([3,5,7,9,11], dtype=float)

model = tf.keras.Sequential([
    tf.keras.layers.Dense(1, input_shape=(2,))
])

model.compile(optimizer='adam', loss='mse')

model.fit(X, Y, epochs=500, verbose=0)

print(model.predict(np.array([[6,7]])))
```

Sample Output

```
[[13.01]]
```

3	**Write a program to convert speech into text.**	COPC 303.2

Aim

To develop a Speech-to-Text system that converts spoken words into textual form using Python and the SpeechRecognition library.

Theory

Speech Recognition, also known as Automatic Speech Recognition (ASR), is a technology that enables a computer to identify and convert spoken language into text. It is widely used in virtual assistants, voice search systems, automated customer support, and transcription services.

The speech recognition process involves the following steps:

1. **Audio Acquisition** – Capturing speech through a microphone or audio file.
2. **Preprocessing** – Removing noise and improving audio quality.
3. **Feature Extraction** – Extracting useful speech features from the audio signal.
4. **Speech Recognition Model** – Processing features using machine learning or deep learning models.

5. **Text Generation** – Producing the corresponding text output.

In this experiment, the SpeechRecognition Python library is used along with Google's Speech Recognition API to convert speech signals into text.

Dataset Used

Speech Commands Dataset v0.02

Description:

The Speech Commands Dataset is provided by TensorFlow and contains approximately 105,000 audio recordings of 35 spoken words such as:

- Yes
- No
- Up
- Down
- Left
- Right
- Stop
- Go

The dataset is commonly used for speech recognition and keyword spotting applications.

Software Requirements

- Google Colab
- Python 3.x
- TensorFlow
- SpeechRecognition Library
- FFmpeg

Algorithm

1. Install required libraries.
2. Download the Speech Commands Dataset.
3. Select a sample audio file from the dataset.
4. Load the audio file using SpeechRecognition.
5. Convert speech to text using Google's Speech Recognition API.
6. Display the recognized text.
7.

Program

```
# Install required libraries
!pip install SpeechRecognition
!apt-get -qq install ffmpeg

import tensorflow as tf
import speech_recognition as sr
import os

# Download Speech Commands Dataset
path = tf.keras.utils.get_file(
    'speech_commands_v0.02.tar.gz',

origin='https://storage.googleapis.com/download.tensorflow
.org/data/speech_commands_v0.02.tar.gz',
```

```
    extract=True
)

# Dataset directory
data_dir = os.path.join(os.path.dirname(path),
'speech_commands_v0.02')

# Select sample audio file
audio_file = os.path.join(data_dir, 'yes',
'0a7c2a8d_nohash_0.wav')

print("Audio File Selected:")
print(audio_file)

# Create recognizer object
recognizer = sr.Recognizer()

# Read audio file
with sr.AudioFile(audio_file) as source:
    audio_data = recognizer.record(source)

# Convert speech to text
try:
    text = recognizer.recognize_google(audio_data)

    print("\nRecognized Text:")
    print(text)

except sr.UnknownValueError:
    print("Could not understand audio")

except sr.RequestError:
    print("Speech Recognition service unavailable")
```

4 **Write a program to convert text into speech.**

COPC 303.2

Theory

Text-to-Speech (TTS) is a technology that converts written text into human-like speech. It is an important application of Artificial Intelligence and Natural Language Processing (NLP). TTS systems are widely used in virtual assistants, navigation systems, screen readers for visually impaired users, and automated customer support systems.

The Text-to-Speech process consists of the following steps:

1. **Text Analysis** – The input text is processed and analyzed.
2. **Text Normalization** – Numbers, abbreviations, and symbols are converted into readable words.
3. **Phoneme Conversion** – Text is converted into phonetic representations.
4. **Speech Synthesis** – The phonemes are converted into speech signals.
5. **Audio Output** – The generated speech is played or saved as an audio file.

In this experiment, the **gTTS (Google Text-to-Speech)** library is used to convert text

	into speech and save it as an MP3 file.
	
	Program
	
	`# Install gTTS`
	`!pip install gtts`
	
	`from gtts import gTTS`
	`from IPython.display import Audio`
	
	`# Input text`
	`text = """`
	`Deep learning is a subset of machine learning that uses neural networks`
	`with multiple layers to learn patterns from data.`
	`"""`
	
	`# Convert text to speech`
	`tts = gTTS(text=text, lang='en')`
	
	`# Save audio file`
	`tts.save("output.mp3")`
	
	`print("Audio file generated successfully!")`
	
	`# Play audio`
	`Audio("output.mp3")`	

5	**Write a program to convert video into frames.** Aim To extract individual frames from a video file using Python and OpenCV. Theory Video processing is an important area of Computer Vision and Deep Learning. A video is a sequence of images called frames displayed at a specific rate known as Frames Per Second (FPS). Extracting frames from a video is a common preprocessing step used in applications such as object detection, activity recognition, video surveillance, and image classification. In this experiment, the OpenCV library is used to read a video file and extract each frame. The extracted frames are saved as image files for further analysis. Applications Object Detection Face Recognition Video Analytics Action Recognition Autonomous Vehicles	COPC 303.3

Software Requirements

Google Colab
Python 3.x
OpenCV
NumPy
Algorithm
Install OpenCV.
Upload a video file.
Read the video using OpenCV.
Extract frames one by one.
Save each frame as an image.
Count the total number of frames extracted.
Display the result.
Dataset Used
Dataset Name

UCF101 Action Recognition Dataset (Sample Video)

Description:
UCF101 is a widely used action recognition dataset containing 13,320 videos from 101 action categories such as walking, running, jumping, and playing sports.

For this experiment, you may use:

Any sample video from UCF101 Dataset, or
A self-recorded MP4 video.

Program

Step 1: Upload Video

```
from google.colab import files

uploaded = files.upload()
```

Upload a video file such as: sample_video.mp4

Step 2: Extract Frames

```
import cv2
import os

# Create folder to save frames
os.makedirs("frames", exist_ok=True)

# Load video
video = cv2.VideoCapture("sample_video.mp4")
```

	count = 0
	
	while True:
	success, frame = video.read()
	
	if not success:
	break
	
	frame_path = f"frames/frame_{count}.jpg"
	cv2.imwrite(frame_path, frame)
	
	count += 1
	
	video.release()
	
	print("Total Frames Extracted:", count)	

6	Write a program for Time-Series Forecasting with the LSTM Model. Aim To implement a Long Short-Term Memory (LSTM) network for time-series forecasting and predict future values based on historical data. Theory Time Series Forecasting is the process of predicting future values using previously observed data points collected over time. Examples include stock prices, weather data, sales forecasting, and electricity consumption prediction. Traditional machine learning algorithms often struggle with sequential data because they cannot effectively remember previous information. To overcome this limitation, Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks are used. LSTM is a special type of RNN that can learn long-term dependencies in sequential data through its memory cells and gates: Forget Gate – Decides which information should be discarded. Input Gate – Decides which new information should be stored. Output Gate – Produces the output based on stored information. LSTM models are widely used for: Stock Market Prediction Weather Forecasting	COPC 303.3

Demand Forecasting
Traffic Prediction
Energy Consumption Prediction

Dataset Link
https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv

Program

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

# Load Dataset
url =
"https://raw.githubusercontent.com/jbrownlee/Datasets/master/air
line-passengers.csv"
data = pd.read_csv(url)

dataset = data['Passengers'].values.reshape(-1,1)

# Normalize Data
scaler = MinMaxScaler(feature_range=(0,1))
dataset = scaler.fit_transform(dataset)

# Create Sequences
X = []
Y = []

for i in range(12, len(dataset)):
    X.append(dataset[i-12:i, 0])
    Y.append(dataset[i, 0])

X = np.array(X)
Y = np.array(Y)

X = np.reshape(X, (X.shape[0], X.shape[1], 1))

# Build LSTM Model
model = Sequential()

model.add(LSTM(50, activation='relu', input_shape=(12,1)))
model.add(Dense(1))

model.compile(optimizer='adam', loss='mse')

# Train Model
model.fit(X, Y, epochs=50, batch_size=8)

# Prediction
predicted = model.predict(X)

predicted = scaler.inverse_transform(predicted)
```

	``
	actual = scaler.inverse_transform(Y.reshape(-1,1))
	
	# Plot Results
	plt.figure(figsize=(10,5))
	plt.plot(actual, label='Actual')
	plt.plot(predicted, label='Predicted')
	plt.legend()
	plt.title("LSTM Time Series Forecasting")
	plt.show()
	``	

7	Write a program to implement deep learning Techniques for image segmentation. Aim To implement image segmentation using a pre-trained deep learning model and identify different regions or objects present in an image. Theory Image Segmentation is a computer vision technique that divides an image into multiple meaningful regions or segments. Unlike image classification, which assigns a single label to an entire image, segmentation classifies each pixel of the image. Segmentation is mainly of three types: Semantic Segmentation – Assigns a class label to every pixel. Instance Segmentation – Identifies individual objects separately. Panoptic Segmentation – Combines semantic and instance segmentation. Deep Learning models such as U-Net, Mask R-CNN, and DeepLabV3 are widely used for image segmentation tasks. Applications include: Medical Image Analysis Autonomous Vehicles Satellite Image Processing Object Detection Industrial Inspection In this experiment, a pre-trained DeepLabV3 model available in TensorFlow Hub is used for image segmentation. Dataset Name Oxford-IIIT Pet Dataset Description	COPC 303.4

	The Oxford-IIIT Pet Dataset contains images of cats and dogs along with segmentation masks. It is commonly used for image segmentation tasks.
	
	# rogram (Google Colab)
	
	Step 1: Install Libraries
	`!pip install tensorflow tensorflow-hub tensorflow-datasets`
	
	--- 	
	
	Step 2: Load Dataset and Perform Segmentation
	`import tensorflow as tf`
	`import tensorflow_datasets as tfds`
	`import matplotlib.pyplot as plt`
	`import numpy as np`
	
	`# Load Oxford Pet Dataset`
	`dataset, info = tfds.load(`
	`'oxford_iiit_pet:3.*.*',`
	`with_info=True`
	`)`
	
	`train_dataset = dataset['train']`
	
	`# Get one sample image`
	`sample = next(iter(train_dataset))`
	
	`image = sample['image']`
	`mask = sample['segmentation_mask']`
	
	`# Display Image and Ground Truth Mask`
	`plt.figure(figsize=(10,5))`
	
	`plt.subplot(1,2,1)`
	`plt.imshow(image)`
	`plt.title("Original Image")`
	
	`plt.subplot(1,2,2)`
	`plt.imshow(tf.squeeze(mask))`
	`plt.title("Segmentation Mask")`
	
	`plt.show()`	

8	Write a program to predict a caption for a sample image using LSTM # Aim To generate a meaningful caption for a given image using a deep learning model based on CNN and LSTM architectures. --- # Theory	COPC 303.4

Image Captioning is a combination of **Computer Vision** and **Natural Language Processing (NLP)**. The objective is to automatically generate a textual description of an image.

The image captioning process involves two major components:

1. **CNN (Convolutional Neural Network)**
   - Extracts visual features from the image.
   - Acts as an image encoder.
2. **LSTM (Long Short-Term Memory)**
   - Generates words sequentially based on extracted image features.
   - Acts as a language decoder.

The workflow is:

- Input Image
- Feature Extraction using CNN
- Feature Vector Generation
- Caption Generation using LSTM
- Output Sentence

Applications include:

- Assistive technology for visually impaired users
- Automatic image annotation
- Social media tagging
- Smart photo management systems

Dataset Name

Flickr8k Dataset

Description

The Flickr8k Dataset contains:

- 8,000 images
- Five captions for each image
- Real-world scenes involving people, animals, sports, and daily activities

It is one of the most commonly used datasets for image caption generation.

Dataset Source

Flickr8k Dataset

Program

Step 1: Install Required Libraries

```
!pip install transformers pillow torch
```

Step 2: Upload Image and Generate Caption

```
from transformers import VisionEncoderDecoderModel
from transformers import ViTImageProcessor
from transformers import AutoTokenizer

from PIL import Image
from google.colab import files
import torch

# Upload Image
uploaded = files.upload()

image_name = list(uploaded.keys())[0]

# Load image
image = Image.open(image_name).convert("RGB")

# Load pretrained image captioning model
model = VisionEncoderDecoderModel.from_pretrained(
    "nlpconnect/vit-gpt2-image-captioning"
)

feature_extractor = ViTImageProcessor.from_pretrained(
    "nlpconnect/vit-gpt2-image-captioning"
)

tokenizer = AutoTokenizer.from_pretrained(
    "nlpconnect/vit-gpt2-image-captioning"
)

# Process image
pixel_values = feature_extractor(
    images=image,
    return_tensors="pt"
).pixel_values

# Generate caption
output_ids = model.generate(
    pixel_values,
    max_length=20
)

caption = tokenizer.decode(
    output_ids[0],
    skip_special_tokens=True
)

print("Generated Caption:")
print(caption)
```

9 Write a program to detect Dog image using YOLO Algorithm

COPC 303.5

Aim

To detect and locate a dog in an image using the YOLO (You Only Look Once) object detection algorithm.

Theory

Object Detection is a computer vision task that identifies and locates objects within an image or video. Unlike image classification, object detection not only classifies objects but also determines their positions using bounding boxes.

YOLO (You Only Look Once) is one of the most popular real-time object detection algorithms. It processes the entire image in a single pass through the neural network, making it faster than traditional object detection methods.

Working of YOLO

1. Input image is divided into a grid.
2. The network predicts bounding boxes and class probabilities.
3. Objects are detected in a single forward pass.
4. Non-Maximum Suppression (NMS) removes duplicate detections.
5. Final detected objects are displayed with bounding boxes.

Advantages of YOLO

- High detection speed
- Real-time object detection
- Good accuracy
- Supports multiple object classes

Applications

- Autonomous Vehicles
- Surveillance Systems
- Face Detection
- Traffic Monitoring
- Animal Detection

Dataset Name

COCO (Common Objects in Context) Dataset

Description

The COCO dataset contains over 330,000 images and 80 object categories, including:

- Dog
- Cat
- Person
- Car
- Bicycle
- Horse
- Bird

	• Bus
	
	YOLO models are pre-trained on the COCO dataset.
	
	# Program
	
	Step 1: Install YOLOv8
	`!pip install ultralytics`
	
	Step 2: Upload Image
	`from google.colab import files`
	
	`uploaded = files.upload()`
	
	Upload an image such as:
	
	`dog.jpg`
	
	Step 3: Dog Detection Using YOLO
	`from ultralytics import YOLO`
	`from PIL import Image`
	`import matplotlib.pyplot as plt`
	
	`# Load YOLOv8 pretrained model`
	`model = YOLO("yolov8n.pt")`
	
	`# Run detection`
	`results = model("dog.jpg")`
	
	`# Display result`
	`results[0].show()`
	
	`# Save output image`
	`results[0].save(filename="dog_detection_result.jpg")`
	
	`print("Dog Detection Completed Successfully")`
	
	Step 4: Display Output Image
	`from PIL import Image`
	`import matplotlib.pyplot as plt`
	
	`img = Image.open("dog_detection_result.jpg")`
	
	`plt.figure(figsize=(8,8))`
	`plt.imshow(img)`
	`plt.axis("off")`
	`plt.show()`	

10	. Write a program for character recognition using RNN and compare it with CNN. # Aim To implement character recognition using a Recurrent Neural Network (RNN)	COPC 303.5

and compare its performance with a Convolutional Neural Network (CNN) using the MNIST handwritten digit dataset.

Theory

Character Recognition is the process of identifying and recognizing handwritten or printed characters from images. It is a major application of Deep Learning and Optical Character Recognition (OCR).

Recurrent Neural Network (RNN)

RNNs are neural networks designed to process sequential data. They maintain information from previous inputs using internal memory. In character recognition, image rows can be treated as sequences and processed by the RNN.

Convolutional Neural Network (CNN)

CNNs are specialized neural networks designed for image processing. They automatically extract spatial features from images using convolution and pooling operations.

Comparison of RNN and CNN

Feature	RNN	CNN
Data Type	Sequential Data	Image Data
Memory	Uses Previous Information	No Internal Memory
Speed	Slower	Faster
Accuracy on Images	Moderate	High
Best Use	Text, Speech, Time Series	Images, Videos

Applications

- Handwritten Digit Recognition
- OCR Systems
- Document Processing
- Bank Check Reading
- License Plate Recognition

Dataset Name

MNIST Handwritten Digit Dataset

Description

The MNIST dataset contains:

- 70,000 handwritten digit images
- Digits from 0 to 9
- 28 × 28 grayscale images

Dataset Split:

- Training Images: 60,000
- Testing Images: 10,000

MNIST is one of the most popular datasets for character recognition and deep learning experiments.

Program (Google Colab)

Step 1: Import Libraries and Load Dataset

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, SimpleRNN
from tensorflow.keras.layers import Conv2D, MaxPooling2D,
Flatten
from tensorflow.keras.utils import to_categorical

# Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)
```

Part A: Character Recognition Using RNN

```
# Build RNN Model
rnn_model = Sequential([
    SimpleRNN(128, input_shape=(28,28)),
    Dense(10, activation='softmax')
])

rnn_model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Train Model
rnn_model.fit(
    X_train,
    y_train_cat,
    epochs=5,
```

```
    batch_size=128,
    validation_split=0.1
)

# Evaluate
rnn_loss, rnn_acc = rnn_model.evaluate(X_test, y_test_cat)

print("RNN Accuracy:", rnn_acc)
```

Part B: Character Recognition Using CNN

```
# Reshape Data
X_train_cnn = X_train.reshape(-1,28,28,1)
X_test_cnn = X_test.reshape(-1,28,28,1)

# Build CNN Model
cnn_model = Sequential([
    Conv2D(32,(3,3),activation='relu',
           input_shape=(28,28,1)),
    MaxPooling2D((2,2)),
    Flatten(),
    Dense(128,activation='relu'),
    Dense(10,activation='softmax')
])

cnn_model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Train Model
cnn_model.fit(
    X_train_cnn,
    y_train_cat,
    epochs=5,
    batch_size=128,
    validation_split=0.1
)

# Evaluate
cnn_loss, cnn_acc = cnn_model.evaluate(
    X_test_cnn,
    y_test_cat
)

print("CNN Accuracy:", cnn_acc)
```